



LeetCode 1929: Concatenation of Array

1. Problem Title & Link

Title: LeetCode 1929: Concatenation of Array

Link: <https://leetcode.com/problems/concatenation-of-array/>

2. Problem Statement (Short Summary)

Given an integer array `nums` of length `n`.

Create a new array `ans` of length `2n` such that:

`ans[i] = nums[i]`

`ans[i + n] = nums[i]`

for every $0 \leq i < n$.

In simple words:

Append the array to itself once.

Return the resulting array.

3. Examples (Input → Output)

Example 1

Input:

`nums = [1,2,1]`

Output:

`[1,2,1,1,2,1]`

Explanation:

Original : `[1,2,1]`

Copy : `[1,2,1]`

Combine : `[1,2,1,1,2,1]`

Example 2

Input:

`nums = [1,3,2,1]`

Output:

`[1,3,2,1,1,3,2,1]`

Example 3

Input:

`nums = [5]`

Output:



[5,5]

4. Constraints

$1 \leq \text{nums.length} \leq 1000$

$1 \leq \text{nums}[i] \leq 1000$

Requirements:

Return a new array of size $2n$.

5. Core Concept (Pattern / Topic)

Array Manipulation

Secondary Topics:

- Traversal
- Array Construction

6. Thought Process (Step-by-Step Explanation)

Brute Force Thinking

We need:

$\text{nums} + \text{nums}$

For every element:

Take it once

Take it again

Store into answer array.

Example:

$\text{nums} = [1, 2, 1]$

Desired:

$[1, 2, 1, 1, 2, 1]$

Observe:

First half = original array

Second half = same array

Optimized Thinking

Create array of size:

$2 * n$

For each index:

$\text{ans}[i] = \text{nums}[i]$



`ans[i+n] = nums[i]`

Done.

No complex logic needed.

7. Visual / Intuition Diagram (ASCII Diagram)

Input:

`nums = [1,2,1]`

Index Mapping:

`n = 3`

`ans[0] = nums[0] = 1`

`ans[3] = nums[0] = 1`

`ans[1] = nums[1] = 2`

`ans[4] = nums[1] = 2`

`ans[2] = nums[2] = 1`

`ans[5] = nums[2] = 1`

Result:

`[1,2,1,1,2,1]`

Visualization:

`nums`

`[1][2][1]`

Copy

`[1][2][1]`

Combine

`[1][2][1][1][2][1]`

8. Pseudocode (Language Independent)

`n = length(nums)`

`create ans of size 2*n`

`for i from 0 to n-1`

`ans[i] = nums[i]`



```
ans[i+n] = nums[i]

return ans
```

9. Code Implementation

Python

Approach 1 (Interview-Friendly)

```
class Solution:
    def getConcatenation(self, nums):

        n = len(nums)

        ans = [0] * (2 * n)

        for i in range(n):

            ans[i] = nums[i]
            ans[i + n] = nums[i]

        return ans
```

Python Shortcut

```
class Solution:
    def getConcatenation(self, nums):
        return nums + nums
```

Java

```
class Solution {

    public int[] getConcatenation(int[] nums) {

        int n = nums.length;

        int[] ans = new int[2 * n];

        for(int i = 0; i < n; i++) {

            ans[i] = nums[i];
            ans[i + n] = nums[i];
        }

    }

}
```



```
    }  
  
    return ans;  
}  
}
```

10. Time & Space Complexity

Time Complexity

$O(n)$

Reason:

One traversal of the array.

Space Complexity

$O(n)$

Reason:

New array of size $2n$ created.

11. Common Mistakes / Edge Cases

Mistake 1

Using:

`ans[i+n+1]`

instead of:

`ans[i+n]`

causes index errors.

Mistake 2

Creating answer size:

`n`

instead of:

`2*n`

Edge Cases

Single Element

Input:

`[7]`

Output:

`[7,7]`



All Same Values

Input:

[5,5,5]

Output:

[5,5,5,5,5,5]

12. Detailed Dry Run (Step-by-Step Table)

Input:

nums = [1,2,1]

n = 3

Initial:

ans = [0,0,0,0,0,0]

i	nums[i]	ans[i]	ans[i+n]
0	1	1	1
1	2	2	2
2	1	1	1

Final:

[1,2,1,1,2,1]

13. Common Use Cases (Real-Life / Interview)

Data Duplication

Repeat data blocks

Circular Buffer Simulation

Repeated sequence generation

Playlist Repetition

Songs repeated twice

Testing Array Operations

Common beginner array interview problem.

14. Common Traps (Important!)

Trap 1

Confusing:

Append array



with

Append elements one by one

Trap 2

Wrong answer size.

Need:

$2 * n$

not:

n

Trap 3

Off-by-One Error

Wrong:

$i+n+1$

Correct:

$i+n$

15. Builds To (Related LeetCode Problems)

Progression:

LC 1929 - Concatenation of Array



LC 1920 - Build Array from Permutation



LC 1480 - Running Sum of 1D Array



LC 867 - Transpose Matrix



LC 1470 - Shuffle the Array

These strengthen array indexing skills.

16. Alternate Approaches + Comparison

Approach	Time	Space	Interview Rating
Extra Array	$O(n)$	$O(n)$	Best
Python nums+nums	$O(n)$	$O(n)$	Pythonic
Dynamic Append	$O(n)$	$O(n)$	Acceptable

Dynamic Append

`ans = []`



```
for num in nums:  
    ans.append(num)
```

```
for num in nums:  
    ans.append(num)
```

Works but uses two loops.

17. Why This Solution Works (Short Intuition)

Each element appears twice in the final array:

Original Position

+

Position shifted by n

Thus every value is copied exactly two times, producing the required concatenated array.

18. Variations / Follow-Up Questions

Follow-Up 1

Repeat array 3 times.

Example:

[1,2]

Output:

[1,2,1,2,1,2]

Follow-Up 2

Repeat array k times.

Example:

k = 5

Follow-Up 3

Concatenate in reverse order.

Input:

[1,2,3]

Output:

[1,2,3,3,2,1]

Follow-Up 4

Can this be done in-place?



Discuss why increasing array size generally requires extra memory.

Interview Takeaway

This problem is simple, but it teaches a very important skill:

Index Mapping

The core idea is:

$\text{ans}[i] = \text{nums}[i]$

$\text{ans}[i + n] = \text{nums}[i]$

Many array interview problems use this same index-shifting technique.